

PAPER • OPEN ACCESS

Simulating quantum circuits with ZX-calculus reduced stabiliser decompositions

To cite this article: Aleks Kissinger and John van de Wetering 2022 *Quantum Sci. Technol.* **7** 044001

View the [article online](#) for updates and enhancements.

You may also like

- [The ZX-calculus is complete for stabilizer quantum mechanics](#)
Miriam Backens
- [Transversal diagonal logical operators for stabiliser codes](#)
Mark A Webster, Armanda O Quintavalle and Stephen D Bartlett
- [Graphical structures for design and verification of quantum error correction](#)
Nicholas Chancellor, Aleks Kissinger, Stefan Zohren et al.

Quantum Science and Technology



PAPER

OPEN ACCESS

RECEIVED
16 November 2021

REVISED
2 March 2022

ACCEPTED FOR PUBLICATION
11 March 2022

PUBLISHED
21 July 2022

Original content from
this work may be used
under the terms of the
[Creative Commons
Attribution 4.0 licence](#).

Any further distribution
of this work must
maintain attribution to
the author(s) and the
title of the work, journal
citation and DOI.



Simulating quantum circuits with ZX-calculus reduced stabiliser decompositions

Aleks Kissinger^{1,*} and John van de Wetering^{1,2}

¹ Department of Computer Science, University of Oxford, Oxford, United Kingdom

² Institute of Computing and Information Sciences, Radboud University, Nijmegen, the Netherlands

* Author to whom any correspondence should be addressed.

E-mail: aleks.kissinger@cs.ox.ac.uk

Keywords: stabiliser theory, classical simulation, ZX-calculus

Abstract

We introduce an enhanced technique for strong classical simulation of quantum circuits which combines the ‘sum-of-stabilisers’ method with an automated simplification strategy based on the ZX-calculus. Recently it was shown that quantum circuits can be classically simulated by expressing the non-stabiliser gates in a circuit as magic state injections and decomposing them in chunks of 2–6 states at a time, obtaining sums of (efficiently-simulable) stabiliser states with many fewer terms than the naive approach. We adapt these techniques from the original setting of Clifford circuits with magic state injection to generic ZX-diagrams and show that, by interleaving this ‘chunked’ decomposition with a ZX-calculus-based simplification strategy, we can obtain stabiliser decompositions that are many orders of magnitude smaller than existing approaches. We illustrate this technique to perform exact norm calculations (and hence strong simulation) on the outputs of random 50- and 100-qubit Clifford + T circuits with up to 70 T-gates as well as a family of hidden shift circuits previously considered by Bravyi and Gosset with over 1000 T-gates.

1. Introduction

Classical simulation of quantum circuits has a variety of applications, from verifying the correct behaviour of quantum hardware and software to general-purpose simulations of quantum many-body systems. The state of the art in classical simulation also determines whether we can consider a quantum computation to actually give some quantum advantage. For instance, while Google’s quantum supremacy result [4] was originally believed to take 10 000 years to reproduce on a classical computer, improvements in simulation techniques reduced this to about 20 days [24] and later to 5 days [36].

Classical simulation, or emulation, of quantum computations is widely believed to be a hard problem, requiring exponential classical resources. However, there are a variety of different approaches to classical simulation whose time and space requirements vary widely based on the size, shape, contents, or required fidelity of the quantum circuit to be simulated. For example, tensor-network based techniques scale exponentially not with the number of qubits, but with the treewidth of the underlying graph of the circuit [34]. Additionally, noisy quantum circuits are generally easier to simulate [19]. Recently, methods based on stabiliser rank [9] and stabiliser extent [8] have been used to simulate circuits which are suitably close to Clifford circuits.

Thanks to the Gottesman–Knill theorem [2], one can efficiently simulate computational basis measurements on the family of quantum stabiliser states, i.e. those states arising from the $|0 \dots 0\rangle$ basis state by means of CNOT, H, and S gates. It follows immediately from the linearity of the Born rule that one can efficiently simulate any state that is expressed as a linear combination of stabiliser states, as long as the number of terms in the decomposition is not too large. The smallest number of stabiliser terms required to express a given state is known as its *stabiliser rank*. While this number is hard to compute in general, heuristics for computing stabiliser decompositions can upper-bound this quantity and can lead to significant speedups in certain families of circuits [8]. In particular, when we have a quantum circuit

consisting of Clifford gates and t non-Clifford T gates, then we can represent this as a sum of $2^{\alpha t}$ Clifford circuits for a parameter $\alpha < 1$ which depends on the decomposition strategy. Smaller values of α typically mean we can simulate circuits with larger numbers of T-gates. For instance, in [9] they simulated a 40-qubit hidden shift algorithm with about 50 T gates in a couple of hours on a consumer laptop. Their method was improved upon in [8] where they could simulate a 50-qubit circuit with 64 T gates. That work used a decomposition strategy where $\alpha \approx 0.48$, because they decompose a group of 6 T magic states into a sum of seven stabiliser states, hence obtaining $7^{t/6} \approx 2^{0.48t}$ terms [11]. Using this ‘chunking’ of T magic states, decompositions have been found with α as low (asymptotically) as 0.3963 [39].

In this paper we improve upon this technique by combining stabiliser decomposition with a ZX-calculus-based simplification strategy previously applied to circuit optimisation in [18, 29]. Any quantum circuit, or more generally, any linear map between qubits, can be represented as a special kind of tensor network called a ZX-diagram. The ZX-calculus [12, 13] then refers to a set of graphical rewrite rules which can be used to transform and simplify ZX-diagrams.

When we translate quantum circuits into ZX-diagrams, the Clifford vs non-Clifford structure is retained. Namely, Clifford gates become diagrams of *Clifford spiders*, i.e. nodes whose phase parameters are integer multiples of $\frac{\pi}{2}$. We can then treat the remaining non-Clifford spiders analogously to the magic states in reference [9] by decomposing them as sums over diagrams of Clifford spiders. Once all non-Clifford spiders are removed, quantum amplitudes can be calculated efficiently by fully simplifying the diagram using a particular rewriting strategy. Furthermore, at each step of the decomposition, we can use this strategy to partially simplify each of the resulting ZX-diagrams as much as possible, enabling non-Clifford spiders to combine and/or cancel out with each other. As we will see, the resulting reduction of non-Clifford spiders can drastically decrease the number of terms in the final stabiliser decomposition.

In this work we focus on Clifford + T circuits, so the non-Clifford spiders are all *T-spiders*, i.e. spiders whose phase angle is an odd multiple of $\frac{\pi}{4}$. Our simplest method, which computes single amplitudes of a circuit, proceeds as follows:

- (a) Write the circuit, together with the desired input state and measurement effect as a ZX-diagram.
- (b) Simplify the diagram as much as possible using the ZX-calculus.
- (c) Pick a set of the non-Clifford spiders and decompose them, obtaining multiple diagrams with fewer non-Clifford spiders.
- (d) Apply the previous two steps recursively to each of the diagrams until no non-Clifford spiders remain, in which case each diagram is simplified to a single complex number.
- (e) The sum of these numbers gives the overall amplitude.

Notably, the simplification in step 2 is applied not just to the overall circuit, but eagerly at each step of the decomposition. As we will see in section 4, this can drastically reduce the number of non-Clifford spiders that need to be decomposed, decreasing the size of the sum in step 5 by many orders of magnitude.

Aside from this heuristic benefit, this method also provides a polynomial boost in performance when compared to stabiliser decomposition with standard (i.e. tableau-based) simulation of each of the summands. For calculating a single pure amplitude, our method runs in $O(N^3 + 2^{\alpha t} t^2)$, where N is the total number of gates in the original circuit. Note that this has no dependence on the number of qubits of the circuit. Unless t is very low, this is dominated by the $O(2^{\alpha t} t^2)$ term. In [9], the authors found a corresponding complexity of $O(2^{\alpha t} t^3)$.

Our improvement comes from the fact that we apply simplifications as early as possible in the decomposition process, so each time we decompose the diagram further, only a constant-sized subdiagram must be further simplified. Using the ZX-calculus, ‘simulating’ a Clifford ZX-diagram amounts to fully simplifying it using the strategy in section 2.3. Hence, applying simplifications early can be seen as a sort of ‘partial evaluation’ of an almost-Clifford ZX-diagram, where the efficiently-simulable parts are calculated in advance, saving some redundant work later.

Calculating marginal probabilities amounts to computing the norm of a state after some post-selected effects have been applied. To do this, we employ the technique of *doubling* a ZX-diagram [14, 41]. This doubles the amount of non-Clifford spiders in a diagram, so it could in principle have a catastrophic effect on performance. However, we find in practice that many of the additional non-Clifford spiders in this doubled diagram cancel out during simplification, yielding a technique that performs very well even compared to more sophisticated methods for estimating the norm. A pleasant side-effect is that our rewriting-based simulation procedure enables us to compute probabilities exactly as expressions of the form $\frac{1}{2^k}(x + y\sqrt{2})$ for $k, x, y \in \mathbb{Z}$, as opposed to floating point numbers, which would be subject to rounding errors.

We benchmark our technique on two classes of circuits: random circuits built out of exponentiated Pauli gates and the hidden shift circuits of reference [8]. We find that we can simulate 50- and 100-qubit exponentiated Pauli circuits with up to 70 T gates, and 50-qubit hidden shift circuits with up to 1400 T gates. This is a significant improvement over the previous largest circuits (in terms of T-count) that have been simulated using the stabiliser decomposition technique. For instance, in reference [8], they only simulated a hidden shift circuit with up to 64 T gates.

We cover the material that is needed for our paper in section 2, in particular, the theory of simulating quantum circuits using stabiliser decompositions, ZX-diagrams, and the ZX-calculus simplification strategy of reference [29]. Then in section 3 we describe our simulation technique, and in section 4 we present our results when applying this technique to our benchmark circuits. We end with some discussion on possible future improvements to our technique in section 5.

2. Preliminaries

2.1. Stabiliser decompositions

The Gottesman–Knill theorem says that we can efficiently simulate a quantum computation as long as the starting state is a stabiliser state, the unitary applied is a Clifford unitary, and the measurement we are doing is a stabiliser measurement [2]. This holds both for *weak* simulation, where we are merely trying to sample from the output distribution of the quantum computation, as well as for *strong* simulation, where we want to determine (possibly up to some small error) the probability of a particular (marginal) measurement outcome.

Concretely, given a Clifford n -qubit unitary U we can determine amplitudes $a_{x_1 \dots x_n} := \langle x_1 \dots x_n | U | 0 \dots 0 \rangle$ in $O(n^3)$ time. It immediately follows that we can obtain probabilities of individual measurement outcomes via the Born rule, i.e. $P(x_1 \dots x_n) = |a_{x_1 \dots x_n}|^2$. The Gottesman–Knill theorem also allows us to compute arbitrary marginal probabilities. That is, for $k < n$, we can compute:

$$\begin{aligned} P(x_1 \dots x_k) &= \sum_{x_{k+1} \dots x_n} P(x_1 \dots x_n) \\ &= \sum_{x_{k+1} \dots x_n} \langle 0 \dots 0 | U^\dagger | x_1 \dots x_n \rangle \langle x_1 \dots x_n | U | 0 \dots 0 \rangle \quad (1) \\ &= \langle 0 \dots 0 | U^\dagger (| x_1 \dots x_k \rangle \langle x_1 \dots x_k | \otimes I_{n-k}) U | 0 \dots 0 \rangle \end{aligned}$$

Now suppose we have a Clifford + T circuit. We can then transform it so that every T gate is implemented by a magic-state teleportation and a post-selection, so the computation of an amplitude for this circuit looks like $\langle x_1 \dots x_n 0 \dots 0 | U | 0 \dots 0 T \dots T \rangle$, where the $|0\rangle$'s in the effect correspond to the post-selections, the U is Clifford and $|T\rangle := \frac{1}{\sqrt{2}}(|0\rangle + e^{i\pi/4}|1\rangle)$ is the T magic state. Marginal probabilities can be presented similarly. The stabiliser states span the entire space of quantum states, and hence we can write this tensor product of T magic states in terms of stabiliser states:

$$|T\rangle^{\otimes n} = \sum_{k=1}^{\chi} \lambda_k |\psi_k\rangle. \quad (2)$$

Here χ denotes the *stabiliser rank* of this decomposition, the amount of stabiliser terms needed to write the state. With such a *stabiliser decomposition* of the magic states in hand we can calculate the amplitude of the circuit as

$$\langle x_1 \dots x_n 0 \dots 0 | U | 0 \dots 0 T \dots T \rangle = \sum_{k=1}^{\chi} \lambda_k \langle x_1 \dots x_n 0 \dots 0 | U | 0 \dots 0 \psi_k \rangle. \quad (3)$$

Each of the summands is now just a stabiliser amplitude and hence efficient to calculate. We see then that the efficiency of this method depends on the amount of stabiliser amplitudes we need to calculate, the stabiliser rank.

In general, calculating the minimal stabiliser rank of $|T\rangle^{\otimes n}$ is hard, because the dimension of the search space increases exponentially with n . We can however find a (probably not optimal) decomposition by iterating a decomposition for a small number of magic states. The minimal stabiliser rank of a single T magic state $|T\rangle$ is 2 as it is not a stabiliser state itself, and has a decomposition $|T\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}e^{i\pi/4}|1\rangle$. By taking tensor products of this decomposition we then get a decomposition of $|T\rangle^{\otimes n}$ that has rank 2^n . We can however do better. Note that $|T\rangle \otimes |T\rangle = \frac{1}{2}(|00\rangle + i|11\rangle) + \frac{1}{2}e^{i\pi/4}(|01\rangle + |10\rangle)$, and hence a pair of T magic states also has stabiliser rank 2 (rather than 4). Hence, for an even n we can decompose $|T\rangle^{\otimes n}$ into $2^{n/2}$ terms by decomposing the magic states in pairs. It turns out that there are even more efficient groupings. In

reference [11] it was shown that there is a decomposition of $|T\rangle^{\otimes 6}$ that requires only seven terms. Using this decomposition, n magic states require $7^{n/6} = 2^{\alpha n}$ for $\alpha \approx 0.468$ stabiliser terms. This results in better scaling than we would get with the $2^{0.5n}$ for the pairwise decomposition. By iterating this 6-to-7 decomposition and combining some terms we can get a 12-to-47 decomposition which gives slightly improved exponent of $\alpha \approx 0.463$ [30]. Very recently, a new family of decompositions was found, including a 6-to-6 decomposition, which gives an exponent of ≈ 0.4308 . This family of decompositions go all the way down to an asymptotic limit of ≈ 0.3963 [39]. Our method relies heavily on a ZX-diagram version of the 6-to-7 decomposition from reference [11], whereas we leave incorporating the more recent improved decompositions as future work.

In [8, 9] some techniques are proposed that make the calculation of (3) faster, for instance by Monte-Carlo sampling weighted by the amplitudes $|\lambda_k|$, as well as enhanced techniques for calculating marginal probabilities using t -designs of stabiliser states. We will not consider these methods in this paper, and instead perform strong simulation by exact calculation of the last expression in equation (1) by stabiliser decomposition.

2.2. The ZX-calculus

We will provide a brief overview of the ZX-calculus. For a review see [42], and for a book-length introduction see reference [14].

The ZX-calculus is a diagrammatic language similar to the familiar quantum circuit notation. A ZX-diagram (or simply *diagram*) consists of *wires* and *spiders*. Wires entering the diagram from the left are *inputs*; wires exiting to the right are *outputs*. Given two diagrams we can compose them by joining the outputs of the first to the inputs of the second, or form their tensor product by simply stacking the two diagrams.

Spiders are linear operations which can have any number of input or output wires. There are two varieties: Z spiders depicted as green dots and X spiders depicted as red dots:

$$\begin{aligned} \text{Z spider } \alpha &:= |0\dots 0\rangle\langle 0\dots 0| + e^{i\alpha}|1\dots 1\rangle\langle 1\dots 1| \\ \text{X spider } \alpha &:= |+\dots +\rangle\langle +\dots +| + e^{i\alpha}|-\dots -\rangle\langle -\dots -| \end{aligned}$$

Note that if you are reading this document in monochrome or otherwise have difficulty distinguishing green and red, Z spiders will appear lightly-shaded and X darkly-shaded. The diagram as a whole corresponds to a linear map built from the spiders (and permutations) by the usual composition and tensor product of linear maps. As a special case, diagrams with no inputs represent (unnormalised) state preparations.

Example 2.1. We can immediately write down some simple state preparations and unitaries in the ZX-calculus:

$$\begin{aligned} \text{green circle} &= |0\rangle + |1\rangle = \sqrt{2}|+\rangle \\ \text{red circle} &= |+\rangle + |-\rangle = \sqrt{2}|0\rangle \\ \text{green circle } \alpha &= |0\rangle\langle 0| + e^{i\alpha}|1\rangle\langle 1| = Z_\alpha \\ \text{red circle } \alpha &= |+\rangle\langle +| + e^{i\alpha}|-\rangle\langle -| = X_\alpha \end{aligned}$$

In particular we have the Pauli matrices:

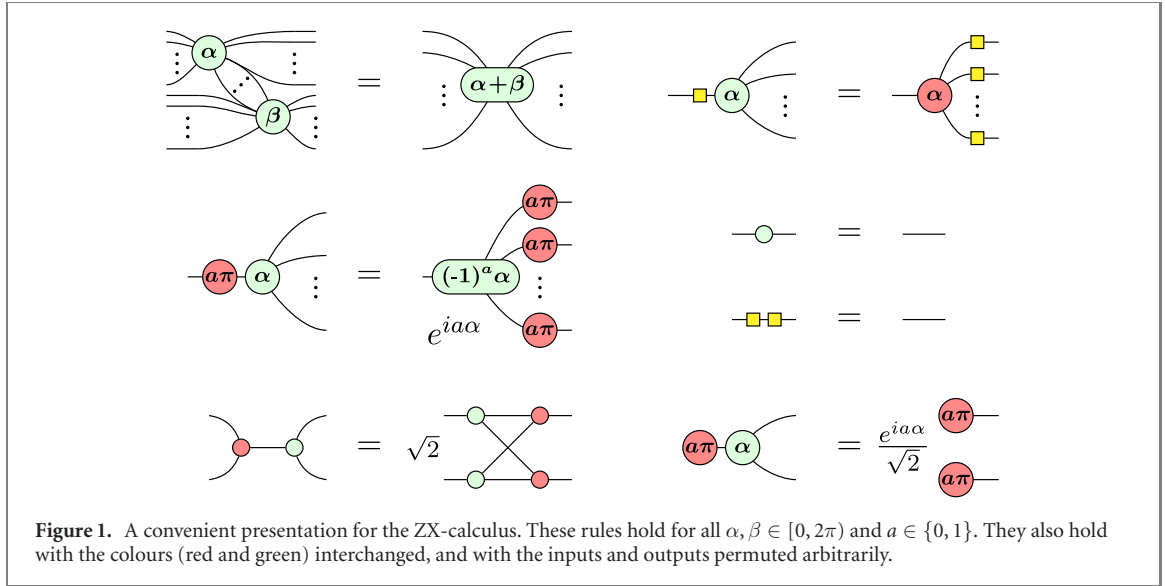
$$\text{green circle } \pi = Z \quad \text{red circle } \pi = X$$

It will be convenient to introduce a symbol—a yellow square—for the Hadamard gate. This is defined by the equation:

$$\text{H} = e^{-i\frac{\pi}{4}} \text{green circle } \frac{\pi}{2} \text{ red circle } \frac{\pi}{2} \text{ green circle } \frac{\pi}{2} =: \text{yellow square} \quad (4)$$

We will often use an alternative notation to simplify the diagrams, and replace a Hadamard between two spiders by a blue dashed edge, as illustrated below.

$$\text{green circle} \text{---} \text{blue dashed line} \text{---} \text{green circle} := \text{green circle} \text{---} \text{yellow square} \text{---} \text{green circle}$$



Both the blue edge notation and the Hadamard box can always be translated back into spiders when necessary. We will refer to the blue edge as a *Hadamard edge*.

Two diagrams are considered *equal* when one can be deformed to the other by moving the vertices around in the plane, bending, unbending, crossing, and uncrossing wires, as long as the connectivity and the order of the inputs and outputs is maintained. Equivalently, a ZX-diagram can be considered as a graphical depiction of a tensor network, as in e.g. [37]. Then, just like any other tensor network, we can observe that the interpretation of a ZX-diagram is unaffected by deformation. As tensors, Z and X spiders can be written as follows:

$$(\text{green circle with } \alpha)_{i_1 \dots i_m}^{j_1 \dots j_n} = \begin{cases} 1 & \text{if } i_1 = \dots = i_m = j_1 = \dots = j_n = 0 \\ e^{i\alpha} & \text{if } i_1 = \dots = i_m = j_1 = \dots = j_n = 1 \\ 0 & \text{otherwise} \end{cases}$$

One can then define X-spiders as Z-spiders conjugated by Hadamard gates or define them explicitly as follows:

$$(\text{red circle with } \alpha)_{i_1 \dots i_m}^{j_1 \dots j_n} = \left(\frac{1}{\sqrt{2}} \right)^{n+m} \cdot \begin{cases} 1 + e^{i\alpha} & \text{if } \bigoplus_{\alpha} i_{\alpha} \oplus \bigoplus_{\beta} j_{\beta} = 0 \\ 1 - e^{i\alpha} & \text{if } \bigoplus_{\alpha} i_{\alpha} \oplus \bigoplus_{\beta} j_{\beta} = 1 \end{cases}$$

where i_{α}, j_{β} range over $\{0, 1\}$ and $\oplus$ is addition modulo 2.

In addition to simple deformations, ZX-diagrams satisfy a set of equations called the ZX-calculus. There exists several variations of the ZX-calculus. The set of rules we will use is presented in figure 1. Diagrams that can be transformed into each other using the rules of the ZX-calculus correspond to equal linear maps. ZX-diagrams with arbitrary angles are expressive enough to represent any linear map [13].

We call linear maps that can be produced by combining Clifford unitaries, stabiliser states and stabiliser post-selections *Clifford maps*. Clifford maps correspond to *Clifford ZX-diagrams*, where all angles are restricted to multiples of $\pi/2$. The rules given in figure 1 are *complete* with respect to linear map equality [5] for Clifford ZX-diagrams. That is, if two Clifford ZX-diagrams describe the same linear map, one can be transformed into the other using the rules in figure 1. Extensions of the calculus exist that are complete for larger families of ZX-diagrams/maps, including *Clifford + T* ZX-diagrams [25], where phases are multiples of $\pi/4$, and arbitrary ZX-diagrams where any phase is allowed [22, 26, 43].

Quantum circuits can be translated into ZX-diagrams in a straightforward manner. We will take as our starting point circuits constructed from the following set of gates, each of which has a convenient representation in terms of spiders:

$$\text{CNOT} = \sqrt{2} \begin{array}{c} \text{green circle} \\ \text{red circle} \end{array} \quad Z_{\alpha} = \text{green circle with } \alpha \quad H = \text{blue square} \quad (5)$$

Note that for the CNOT gate, the green spider is the first (i.e. control) qubit and the red spider is the second (i.e. target) qubit. Other common gates can easily be expressed in terms of these gates. In particular,

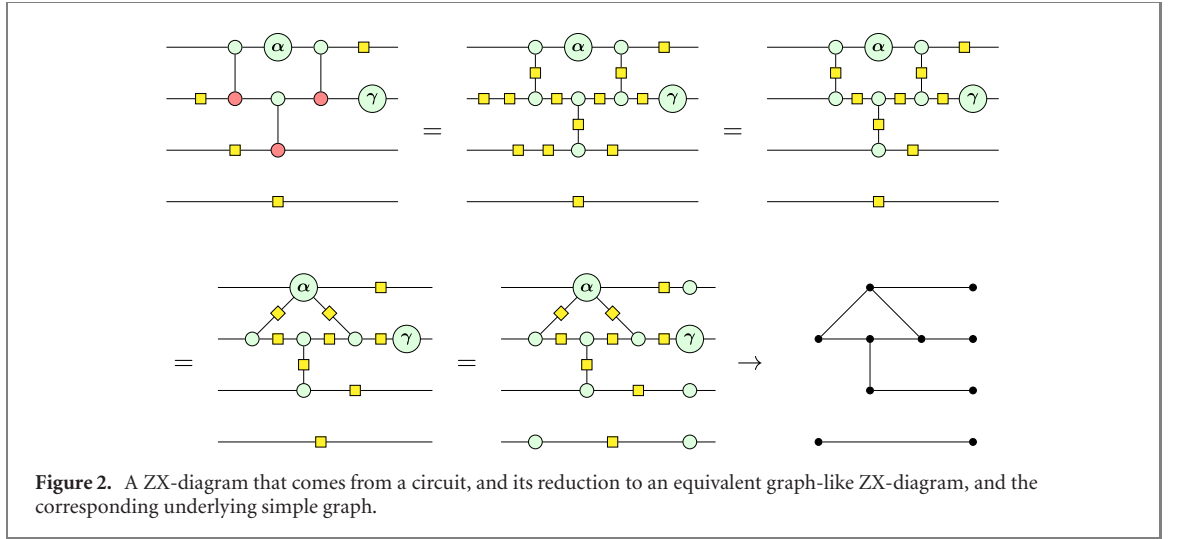


Figure 2. A ZX-diagram that comes from a circuit, and its reduction to an equivalent graph-like ZX-diagram, and the corresponding underlying simple graph.

$S := Z_{\frac{\pi}{2}}$, $T := Z_{\frac{\pi}{4}}$ and:

$$\begin{aligned} X_{\alpha} &= \text{[diagram]} = \text{[diagram]} \\ CZ &\propto \text{[diagram]} = \text{[diagram]} = \text{[diagram]} \end{aligned} \quad (6)$$

For purposes of automated simplification of ZX-diagrams it is often useful to transform the diagram into a *graph-like* ZX-diagram [18].

Definition 2.2. A ZX-diagram is *graph-like* when:

- (a) All spiders are Z-spiders.
- (b) Z-spiders are only connected via Hadamard edges.
- (c) There are no parallel Hadamard edges or self-loops.
- (d) Every input or output is connected to a Z-spider and every Z-spider is connected to at most one input or output.

In reference [18] it is shown that any ZX-diagram can efficiently be transformed into a graph-like ZX-diagram using the rules of the ZX-calculus. This transformation essentially amounts to turning all X spiders into Z spiders with the colour-change rule (top-right in figure 1), fusing as many spiders together as possible, and removing parallel edges/self-loops with the following derived rules:

$$\begin{aligned} \dots \text{[diagram]} \dots &= \frac{1}{2} \dots \text{[diagram]} \dots \\ \text{[diagram]} &= \text{[diagram]} \\ \text{[diagram]} &= \frac{1}{\sqrt{2}} \text{[diagram]} \end{aligned} \quad (7)$$

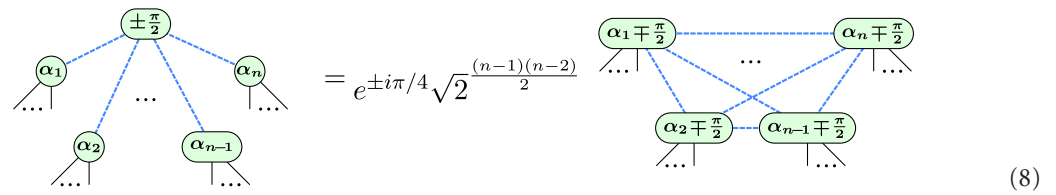
See figure 2 for an example. Note that this procedure never increases the number of non-Clifford phases, but that it can actually decrease the number by fusing spiders, as phases are added together. We call these diagrams *graph-like* because the resulting ZX-diagram is essentially an indirected, simple graph whose vertices are labelled by phase angles.

2.3. Simplifying ZX-diagrams

In this paper we use the simplification strategy for ZX-diagrams described in [29]. We will give a brief summary of this method here. Note that all of the rules in this section can be derived from the basic rules of the ZX-calculus given in figure 1. For the details see [29].

The main idea of this simplification strategy is that we want to remove as many spiders from the diagram as possible. The first step is that we reduce the diagram to graph-like form, so that all the spiders are Z-spiders, and the only connectivity is via Hadamard edges. The main tools for the simplification are two simplification rules called *local complementation* and *pivoting*. The first of these removes an *internal*

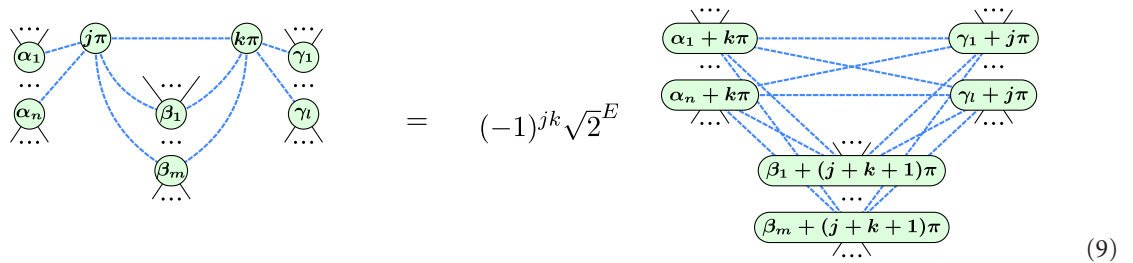
spider (one who is not connected to an input or output wire) with a $\pm \frac{\pi}{2}$ phase, at the cost of introducing a totally connected graph of Hadamard edges on its neighbourhood and changing some phases as follows:



$$= e^{\pm i\pi/4} \sqrt{2}^{\frac{(n-1)(n-2)}{2}} \quad (8)$$

Using equation (7), pairs of Hadamard edges between neighbours of the deleted vertex will actually cancel out, so this has the effect of ‘complementing’ the set of edges involving the neighbourhood of the deleted vertex. While this rule might introduce a lot of new edges between vertices, the important point is that the resulting diagram has one less vertex. Hence, this can only be applied a finite number of times before it terminates.

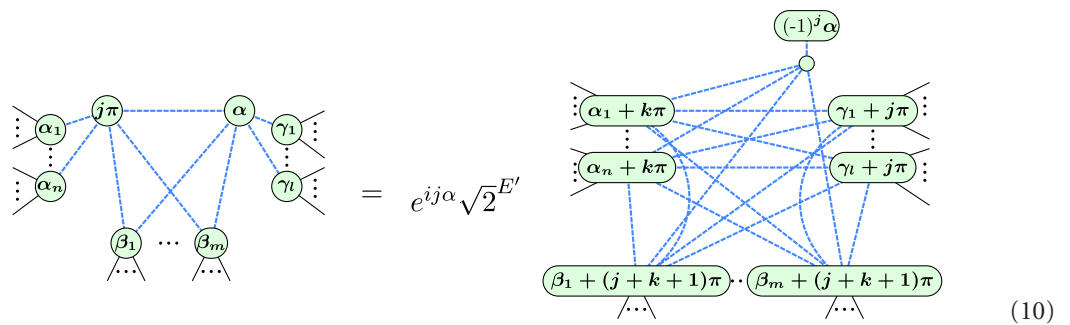
The second rewrite rule removes a pair of connected internal spiders that both have a phase of 0 or π , again at the cost of introducing some new edges. That is, if we have a spider with a phase of $j\pi$ connected to a spider with a phase of $k\pi$, for $j, k \in \{0, 1\}$, we can delete both spiders and connect up their neighbours as follows:



$$= (-1)^{jk} \sqrt{2}^E \quad (9)$$

where $E = (n-1)m + (l-1)m + (n-1)(l-1)$. Here the neighbours of the two deleted vertices fall into three categories: those connected to the left vertex (initially with phases α_i above), those connected to the right vertex (phases γ_i), and those connected to both (phases β_i). When we perform a pivot, a complete bipartite graph of Hadamard edges is introduced between each of these three sets of neighbours and their phases are updated depending on which set they are in and the values of j and k .

These rules suffice to remove almost all internal *Clifford* spiders (those whose phase is a multiple of $\frac{\pi}{2}$). Indeed, if we are dealing with a scalar diagram (for instance if we are calculating an amplitude), then the only Clifford spiders left after applying these rules wherever they can be applied are those spiders with a 0 or π phase that are connected only to *non-Clifford* spiders. Such spiders can also be removed, in a sense, by transforming them into *phase gadgets* using a variation on the pivoting rule [29]:



$$= e^{ij\alpha} \sqrt{2}^{E'} \quad (10)$$

where $E' = E + m + n - 1 = (n-1)m + lm + (n-1)l$.

A phase gadget is simply a phaseless spider to which is attached a one-ary spider:

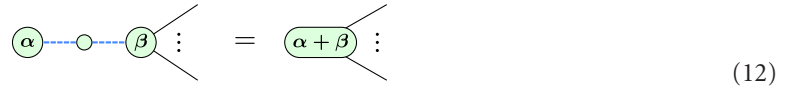


$$\quad (11)$$

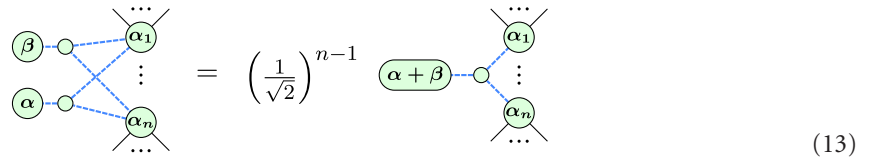
For many purposes it is helpful to consider the two spiders of a phase gadget together as a single component. In particular, for the application of a pivot (9) we do not consider the ‘base’ (i.e. the phase-free spider) in a phase gadget a valid target for pivoting, because doing so could result in a loop of applications of (9) and (10).

So far we have only shown how we can remove all the Clifford spiders by either removing them directly or making them part of a phase gadget. We now show two rewrite rules that apply to phase gadgets that

allow us to merge or kill some non-Clifford phases in the diagram. The first allows us to fuse a phase gadget whose base has only a single neighbour into this neighbour.



The second allows us to merge phase gadgets whose bases have exactly the same neighbourhood:



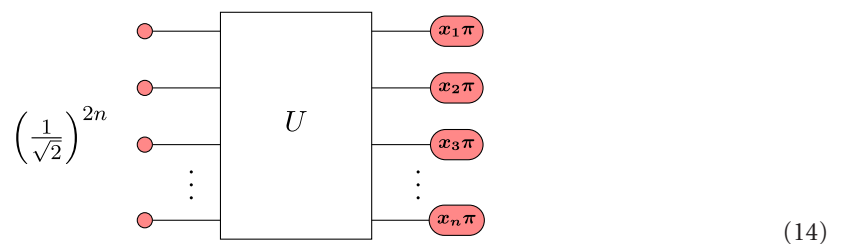
Note that these last rules can combine $\frac{\pi}{4}$ phases together so that they become multiples of $\frac{\pi}{2}$, meaning that they become targets for the application of (8) and (9). We hence repeat this procedure until none of the rewrite rules match. In the case we are dealing with a scalar diagram we will then have a diagram where every spider has a non-Clifford phase or is part of a phase gadget whose phase is non-Clifford. If we started with a circuit with t non-Clifford gates, we will hence get a diagram of size $O(t)$. Note that if the original diagram had N spiders, that this procedure takes at most $O(N^3)$ elementary graph operations (vertex deletions, edge toggles), as each rewrite can affect at most N^2 edges, and we apply at most N of these rewrite rules as each deletes a vertex.

3. Methods

Our method for classically simulating a quantum circuit consists of essentially combining the work done on stabiliser decompositions with that done on ZX-diagram simplification. We first describe the procedure for calculating a single amplitude then proceed to the calculation of marginal probabilities.

Let U be a unitary given as an n -qubit Clifford + T circuit that consists of N gates of which t are T gates. Our goal is to evaluate the amplitude $\langle x_1 \dots x_n | U | 0 \dots 0 \rangle$ for some fixed computational basis effect $\langle x_1 \dots x_n |$.

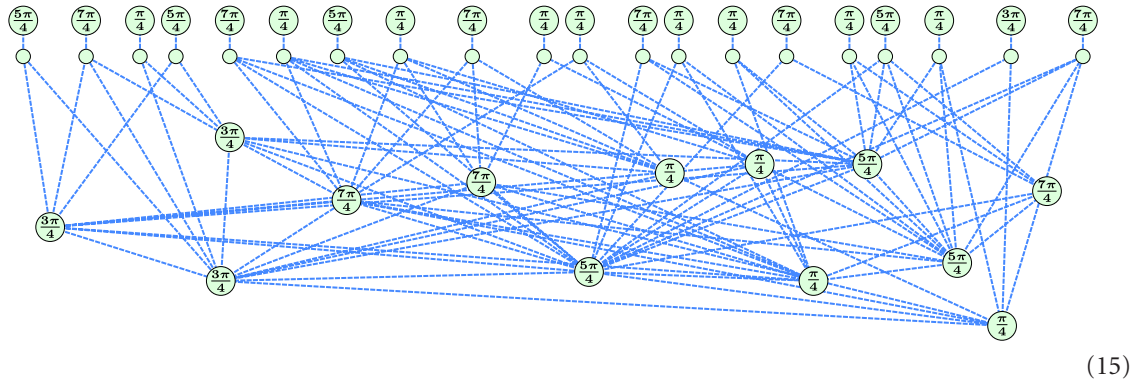
The first step is to write this amplitude as a ZX-diagram. This is straightforward. We simply translate each of the gates in U to spiders and compose the resulting circuit-like ZX-diagram with the necessary basis states and effects $|0\rangle$, $|1\rangle$, $\langle 0|$, and $\langle 1|$ (which can also be expressed as spiders, as described in example 2.1). This results in the following ZX-diagram:



Since this diagram has no inputs or outputs, it represents a $2^0 \times 2^0$ matrix, i.e. a scalar.

We then simplify this diagram using the procedure described in section 2.3. This takes at most $O(N^3)$ elementary graph operations and results in a diagram with $O(t)$ spiders, where each T gate that remains after the simplification contributes either a single spider or a phase gadget. For example, after simplification,

a circuit could yield a reduced ZX-diagram that looks like this:



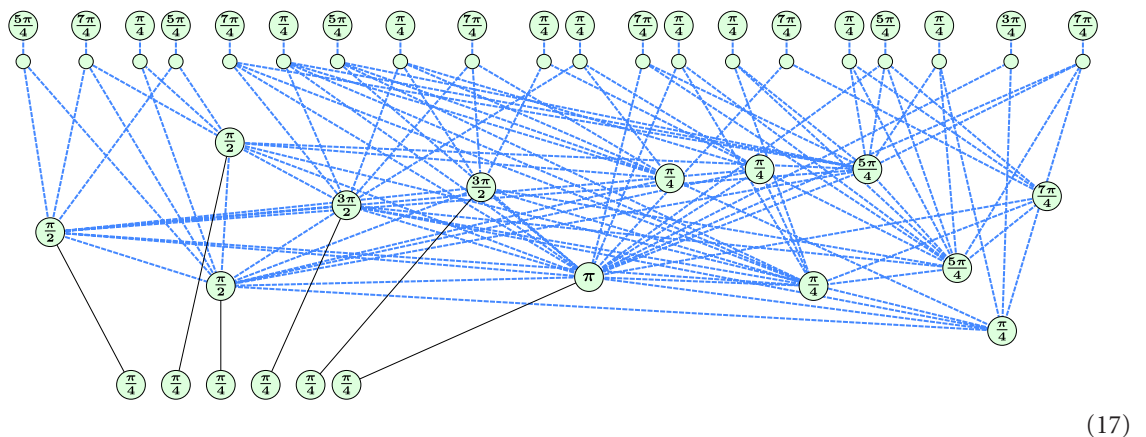
(15)

Now comes the step where the magic happens. In [11], equation (11), Bravyi, Smith, and Smolin gave a decomposition of six $|T\rangle$ magic states into seven stabiliser terms, which we will refer to as the *BSS decomposition*. We can express a ZX-diagram version of the BSS decomposition as follows:

$$\begin{aligned}
 e^{i\pi/4} \begin{array}{c} | \\ \pi/4 \end{array} \begin{array}{c} | \\ \pi/4 \end{array} \begin{array}{c} | \\ \pi/4 \end{array} \begin{array}{c} | \\ \pi/4 \end{array} \begin{array}{c} | \\ \pi/4 \end{array} \begin{array}{c} | \\ \pi/4 \end{array} &= +2e^{i\pi/4} \begin{array}{c} \text{Diagram with 6 nodes and 3 arcs} \\ \pi/2 \end{array} \\
 -\frac{1+\sqrt{2}}{4} \begin{array}{c} | \\ \pi/4 \end{array} \begin{array}{c} | \\ \pi/4 \end{array} \begin{array}{c} | \\ \pi/4 \end{array} \begin{array}{c} | \\ \pi/4 \end{array} \begin{array}{c} | \\ \pi/4 \end{array} \begin{array}{c} | \\ \pi/4 \end{array} &+ \frac{1-\sqrt{2}}{4} \begin{array}{c} | \\ \pi \end{array} \begin{array}{c} | \\ \pi \end{array} \begin{array}{c} | \\ \pi \end{array} \begin{array}{c} | \\ \pi \end{array} \begin{array}{c} | \\ \pi \end{array} \begin{array}{c} | \\ \pi \end{array} \\
 -2\sqrt{2}i \begin{array}{c} \text{Diagram with 6 nodes and 3 arcs} \\ \pi/2 \end{array} &-2i \begin{array}{c} \text{Diagram with 6 nodes and 3 arcs} \\ \pi \end{array} \\
 +8\sqrt{2}i \begin{array}{c} \text{Diagram with 6 nodes and 3 arcs} \\ \pi/2 \end{array} &+8\sqrt{2}i \begin{array}{c} \text{Diagram with 6 nodes and 3 arcs} \\ \pi \end{array}
 \end{aligned}
 \tag{16}$$

This decomposition of ZX-diagrams was obtained by finding an explicit local equivalence between each of the stabiliser summands from [11] to either a product state, a GHZ state, or a more general graph state. Stabiliser states in this form are then readily expressible as ZX-diagrams. The coefficients were adjusted to account for different normalisation conventions. As noted in [11], while the state $|T\rangle^{\otimes 6}$ is itself symmetric, it is not in the space spanned by symmetric *stabiliser* states, necessitating the last two non-symmetric terms in the sum.

Now we pick any six non-Clifford spiders from the diagram and apply the spider-fusion rule (the top-left rule from figure 1) from right-to-left to isolate, or ‘unfuse’, a $\frac{\pi}{4}$ factor from each of these spiders:



(17)

We then apply the decomposition (16) to these six spiders with the $\frac{\pi}{4}$ phase, resulting in seven diagrams where these non-Clifford spiders have been replaced by some Clifford subdiagram. We then perform the necessary spider fusions to reduce each of the diagrams to graph-like form. For each of the diagrams we again apply the simplification algorithm of section 2.3 so that all the new Clifford spiders are removed, and

often some additional non-Clifford spiders are combined as well. Now for each of the diagrams we pick six non-Clifford spiders to apply the decomposition to and we repeat the procedure. If there are not six non-Clifford spiders then the diagram is small enough that we can directly calculate the scalar it represents, or alternatively, we can apply the pairwise stabiliser decomposition to reduce it further.

Once every Clifford spider has been removed from a diagram, we are only left with the scalar factor that arose from application of the simplification rules in section 2.3, which gives us the amplitude of a single term in the decomposition. We compute the overall amplitude by summing these numbers together.

Because we are simplifying the diagrams after each decomposition, we will sometimes see cancellations of non-Clifford phases. This is the primary benefit of our method, as the number of non-Clifford phases in the computation is what leads to the exponential cost of the simulation. However, even if we do not find any cancellations there is an asymptotic benefit to our method.

Suppose none of the non-Clifford phases cancel. Then after applying the magic state decomposition (16) we require a constant number of rewrites to simplify the diagram. As the size of the diagrams is $O(t)$, a rewrite requires at most $O(t^2)$ graph operations (searching for rewrites to apply requires $O(t)$ operations and hence can be ignored). There will be $2^{\alpha t}$ diagrams, all of size at most $O(t)$. Hence, the number of graph operations required is $O(2^{\alpha t} t^2)$. This compares favourably to the $O(2^{\alpha t} t^3)$ found in [11]. Note that the exponential factor is of course of much higher import, and our method performs better in practice not because of this asymptotic improvement, but because we have to do fewer decompositions due to the reduction in the number of non-Clifford phases.

Calculating marginal probabilities is very similar to the calculation of single amplitudes. As we noted before in equation (1), the marginal probability corresponding to measuring the first k qubits can be computed from the state $U|0 \dots 0\rangle$ as follows:

$$P(x_1 \dots x_k) = \langle 0 \dots 0 | U^\dagger (|x_1 \dots x_k\rangle \langle x_1 \dots x_k| \otimes I_{n-k}) U | 0 \dots 0 \rangle.$$

Translating this expression into ZX-calculus notation, we obtain the following ZX-diagram:

$$P(x_1 \dots x_k) =$$

(18)

Note that the state $U|0 \dots 0\rangle$ appears twice: once as itself and once as its adjoint. This is a special case of the technique sometimes referred to as the *CPM-construction* [41] or simply *doubling* [14] in the diagrammatic/categorical quantum mechanics literature.

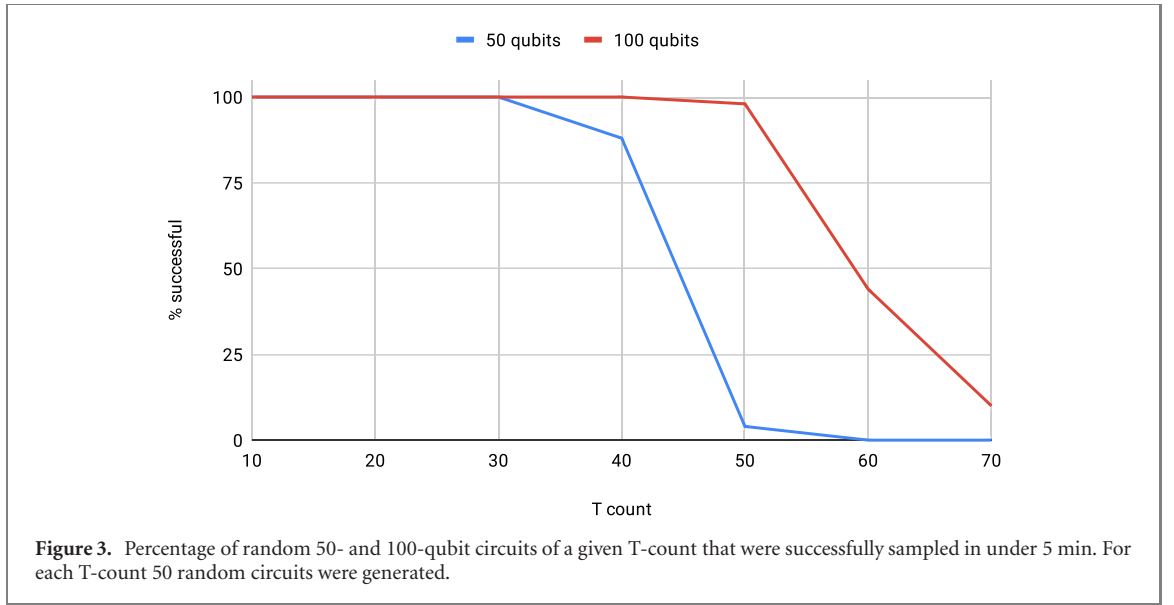
Since we still represent a marginal probability as a single ZX-diagram, we can compute it using the exact same method we described for computing single amplitudes. However, note that we now have both U and $U^\dagger$, each of which contains t non-Clifford gates so that this diagram contains $2t$ non-Clifford gates. Naively this means that calculating a marginal probability will take (exponentially) longer than calculating a single amplitude. However, as U connects directly to $U^\dagger$ on some of the wires, many of the gates on both sides can cancel out, particularly when nearly all of the qubits are being marginalised over. This is in fact what we observe in practice, so that calculating a marginal probability tends to be significantly faster than computing an amplitude of a circuit with twice as many T gates.

3.1. Implementation

We implemented the stabiliser rank decomposition in `quizzx` [1], a port into the Rust programming language of the `PyZX` library for representing and rewriting quantum circuits and ZX-diagrams [28]. Instructions for downloading and building the tool, as well as reproducing the data in the next section, are available at:

<https://github.com/Quantomatic/quizzx/blob/stabrank-v1/stabrank.md>.

The algorithm is implemented as described in the previous section, where ZX-diagrams are represented as sparse undirected graphs. All scalar factors are stored as elements of the ring $\mathbb{D}[e^{i\pi/4}]$, where $\mathbb{D}$ is the ring of dyadic rational numbers. In other words, we store scalars as five integer parameters (k, a, b, c, d) representing the complex number $\frac{1}{2^k}(a + be^{i\pi/4} + ci + de^{-i\pi/4})$. Using this representation, as opposed to floating point complex numbers, we can store the values of any scalars arising from Clifford + T circuits exactly [20]. Probabilities arise as positive real numbers in this ring, which all have the form $\frac{1}{2^k}(x + ye^{i\pi/4} + ye^{-i\pi/4}) = \frac{1}{2^k}(x + y\sqrt{2})$.



To compute amplitudes, we first do a partial decomposition of a ZX-diagram to a small fixed depth d (in our case $d = 3$), then evaluate the amplitude of each of the resulting $K \leq 7^d$ diagrams in parallel. For each of these diagrams, the remaining decomposition is done in a depth-first fashion, so at most $7^d \cdot \lceil \frac{t}{6} \rceil$ diagrams are kept in memory at any given time. This enables us to make use of a multicore system without generating too much memory overhead.

4. Results

To assess the effectiveness of our approach, we performed classical simulation for two families of circuits: (i) random Clifford + T circuits arising from exponentiated Pauli unitaries and (ii) random instances of the hidden shift algorithm. The latter is exactly the family of circuits considered in [9].

4.1. Random Clifford + T circuits

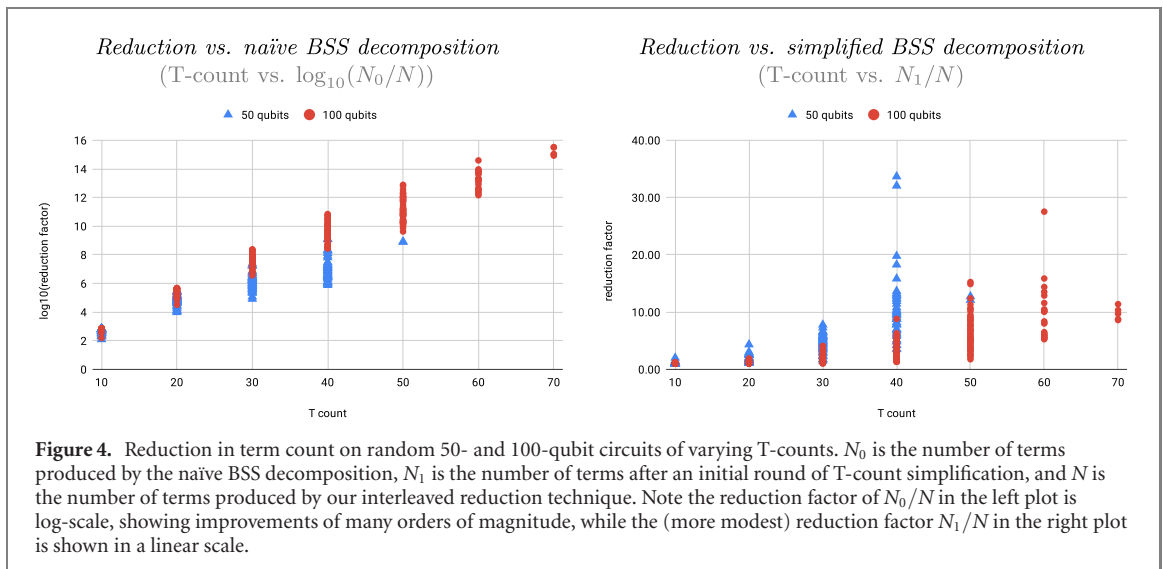
An *exponentiated Pauli unitary* is a unitary of the form $U_{P,\alpha} := \exp(-i\frac{\alpha}{2}P)$ for P an element of the N -qubit Pauli group $P \propto P_1 \otimes \dots \otimes P_n$ for $P_i \in \{I, X, Y, Z\}$. We say $U_{P,\alpha}$ has *weight* k if P contains at most k non-identity Pauli operators. Operators of this form arise naturally in various contexts, including generic representations of Clifford + T circuits [21, 33] and Hamiltonian simulation via trotterisation [15, 16].

We generate random Clifford + T circuits by fixing a number of exponentiated Paulis which will in turn fix the T count. For each exponentiated Pauli $U_{P,\alpha}$, we choose a random Pauli string P with a weight k between some fixed minimum and maximum weights, as well as a random odd integer multiple of $\frac{\pi}{4}$ for α . In this case, $U_{P,\alpha}$ can always be implemented with exactly 1 T gate, $2(k - 1)$ CNOT gates, and local Cliffords [15].

For our experiments, we generated random 50 and 100 qubit circuits with minimum weight 2 and maximum weight 4. Depths (a.k.a. T counts) range from 10 to 70. For each circuit, we use strong simulation to produce one full sample according to the output distribution of the circuit. Namely, we compute the marginal probability $P(q_1 = 0)$ of obtaining outcome 0 on the first qubit, then fix $b_0 \in \{0, 1\}$ probabilistically according to $P(q_1 = 0)$. We then apply $\langle 0|$ or $\langle 1|$ to the first output qubit according to the value of b_1 , compute the probability $P(q_2 = 0|q_1 = b_1)$ in order to select b_2 , and so on until we obtain the full bitstring $b_1, \dots, b_N$ as well as an exact expression for the probability $P(q_1 = b_1, \dots, q_N = b_N)$.

As this is a heuristic method, it is difficult to estimate the runtime, apart from a (typically infeasible) upper bound of $O(N \cdot 7^{2t/6} t^2)$ for N qubits and t T gates, which comes from applying the naïve BSS decomposition to the doubled circuit. So to demonstrate the feasibility of our method, we did numerical experiments. We ran all circuits with a timeout of 5 min on a dedicated computation server (24-core Intel Xeon E5-2667, 2.90 GHz) and computed the percentage that were successfully simulated before the timeout (figure 3).

We found that for random circuits the simulation performed well up to a certain T-count and then the success-rate fell off rapidly, as one would expect from the exponential complexity of the method. For random 50-qubit circuits, we could simulate in less than 5 min all circuits with T-counts up to 30, the majority (88%) of circuits with T-count 40, 4% of circuits with T-count 50, and no circuits with higher



T-counts. What might seem initially counter-intuitive is that 100-qubit circuits actually perform better: all circuits of T-count 40 or less are simulated within 5 min, along with all but one of the T-count 50 circuits, 44% of the T-count 60 circuits, and 10% of the T-count 70 circuits.

To understand this effect, it is worth noting that these circuits with large numbers of qubits are very sparse. For 50-qubit circuits of depth 70, each qubit participates in 4.2 Pauli exponentials on average. For the 100-qubit circuits, each qubit will participate in half as many. Empirically, it seems to be the case that sparser circuits enable the ZX-calculus simplifier to eliminate more T gates, which has a drastic effect on simulation times.

In all cases, the ZX-calculus simplified decomposition produced much lower numbers of stabiliser terms than naïve BSS decompositions of the doubled circuits (figure 4, left). For example, strong simulation with the naïve BSS decomposition on a 50-qubit circuit with 40 T gates requires approximately $50 \times 7^{2 \cdot 40/6} \approx 9 \times 10^{12}$ terms whereas the ZX-calculus simplified decomposition of the random circuits produced an average of 2.6×10^6 terms, an improvement of six orders of magnitude. In the case of higher T-counts, we see in figure 4 (left) improvements of up to 15 orders of magnitude.

It is natural to ask how much of this effect is due to the interleaved ZX-calculus simplifications, and how much comes from the initial simplification of the doubled circuit, where many gates may cancel and the T-count might otherwise be reduced before we begin. To see the effect of interleaving the simplification, we can compare our technique to doing a full BSS decomposition not on the original doubled circuit, but on one that has already been optimised for T-count. For random circuits, we still get improved performance, but it is not nearly as striking. In figure 4 (right), we see about a 10X reduction on average in the number of stabiliser terms if we interleave simplifications, with a peak of about 35X. In this regime of essentially unstructured circuits, it may therefore be more effective to do T-count reduction in advance and then hand off the resulting (post-selected) circuit to a fast tableau simulator.

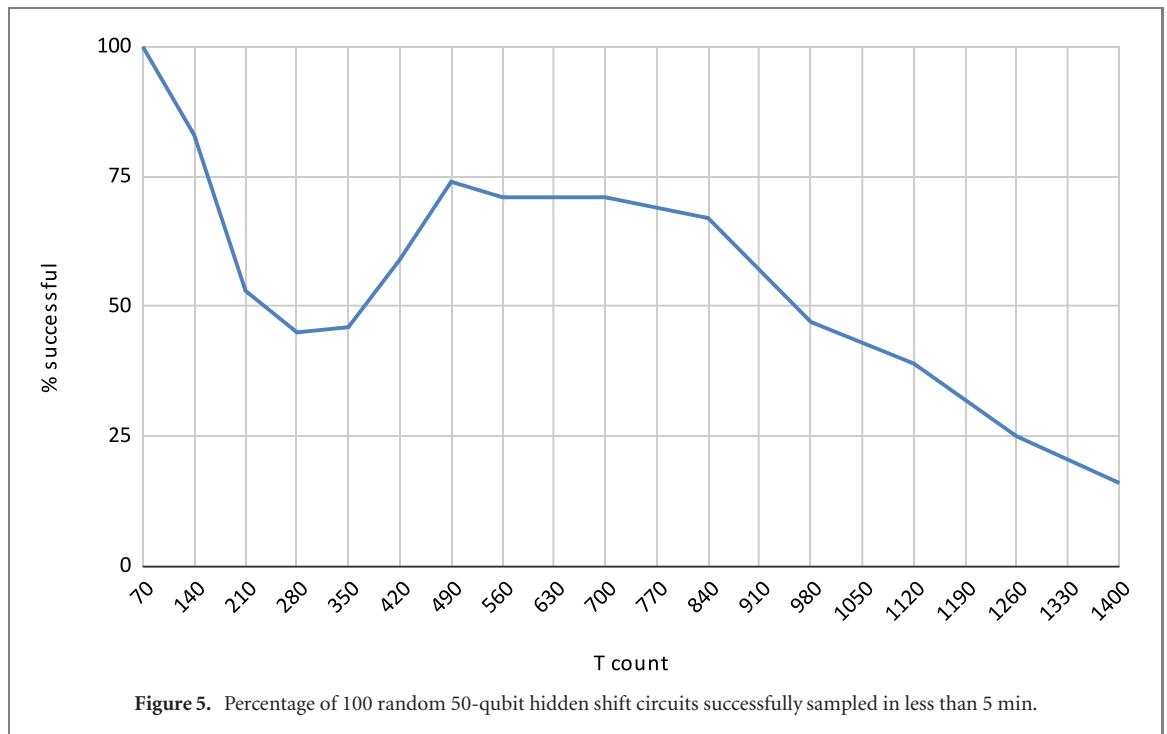
We will see in the next section that for structured circuits, this story can be very different.

4.2. Hidden shift circuits

We will now apply our simulation procedure to random 50-qubit instances of the hidden shift algorithm. This is the same family of circuits considered in [8, 9]. A detailed description of the generation of these circuits can be found in the supplemental material of the latter³. Briefly, each circuit contains two classical oracles which together ‘hide’ a bitstring in their relationship with each other, which can be recovered deterministically from a single query to each oracle using a simple quantum algorithm [40].

There are several notable properties for this family of circuits, which make them a good testbed. The first is that we can define the classical oracle such that the hidden shift is known, so we can always check that the simulator gets the right answer. The second is that, since the hidden shift algorithm gives a deterministic outcome, any joint distribution is a product of the single-qubit marginals. Hence, it is only necessary to compute single-qubit marginals to fully simulate the circuit. Finally, by varying the number of

³ Note the published version contains an error which has been corrected in the arXiv preprint (v3) [10].



CCZ gates which appear in the classical oracles, we can precisely control the number of T gates which occur in the resulting circuit.

The hidden shift circuits are initially generated in the gate set H, Z, CZ, CCZ. To produce a ZX-diagram for this circuit, we must translate each of these gates into ZX-diagrams. The ZX-diagrams for H, Z, and CZ were already introduced in section 2.2. For CCZ, we have (at least) two useful encodings into a ZX-diagram: one that has seven non-Clifford spiders and one that has four non-Clifford spiders. The seven-spider version is the following:

$$\begin{array}{c} \bullet \\ \bullet \\ \bullet \end{array} = \sqrt{2}^5 \begin{array}{c} \text{Diagram with 7 spiders} \end{array} \quad (19)$$

The diagram on the right of equation (19) shows a ZX-diagram for the CCZ gate using seven non-Clifford spiders (green circles). It consists of three horizontal lines (qubits). The top line has a spider at the top. The middle line has two spiders, one on the left and one on the right. The bottom line has two spiders, one on the left and one on the right. The spiders are connected by blue lines forming a complex web. The phase of each spider is $\pi/4$.

It arises from simplifying the ‘textbook’ presentation of CCZ or Toffoli in terms of CNOT and T gates (see e.g. [35], section 4.3) or from the graphical Fourier transform [31, 42]. The four-spider version is the following:

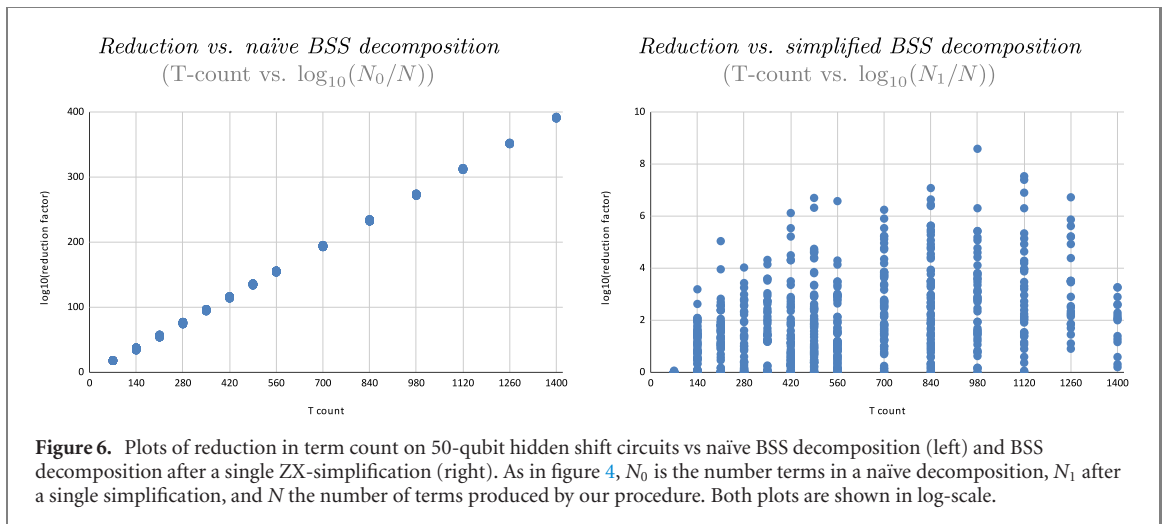
$$\begin{array}{c} \bullet \\ \bullet \\ \bullet \end{array} = 4e^{i\frac{\pi}{4}} \begin{array}{c} \text{Diagram with 4 spiders} \end{array} \quad (20)$$

The diagram on the right of equation (20) shows a ZX-diagram for the CCZ gate using four non-Clifford spiders (green circles). It consists of three horizontal lines (qubits). The top line has a spider at the top. The middle line has two spiders, one on the left and one on the right. The bottom line has a spider at the bottom. The spiders are connected by blue lines forming a complex web. The phase of each spider is $\pi/4$, except for the bottom spider which has a phase of $\pi/2$.

It can be derived by simplifying the (postselected version of the) CCZ gadget introduced by Jones [27]. Either can be verified by concrete calculation.

While the second decomposition results in lower initial T-counts, the first enables the ZX-calculus simplifier to find many more rewrites. For example, the simplification strategy from section 2.3 is able to cancel out the composition of two copies of (19), but not (20). For this reason, we choose the first decomposition for translating hidden shift circuits into ZX-diagrams.

As before, we fix a 5 min timeout for producing a single full sample of each circuit and compute the percentage of circuits that were successful (figure 5). For each given CCZ count we generated 100 random, 50-qubit hidden shift instances and computed each’ single-qubit marginals of all the qubits. We vary the CCZ counts of the circuits from 10 to 80 in increments of 10, then 100 to 200 in increments of 20. Reported T-counts in figures 5 and 6 are then $7 \times$ CCZ count.



As mentioned above, this is a deterministic algorithm, so we fix the outcome on the i th qubit according to this marginal, i.e. $b_i = 0$ if $P(q_i = 0) = 1$ and $b_i = 1$ otherwise. We then verified for all successfully-simulated circuits that this indeed matches the hidden shift between the two classical oracles.

For a CCZ count of 10 (T-count 70) we observe 100% success, then some fluctuation in the success rate until a CCZ count of 120 (T-count 840), followed by a much more gradual drop in success rate as compared with the random circuits. We were successful in simulating the largest family of circuits (200 CCZ/1400 T) 17% of the time.

Interestingly, unlike the random circuit case, interleaving ZX simplification still produced substantial benefits vs a single round of T-count optimisation (figure 6), yielding many reductions in stabiliser terms of more than six orders of magnitude.

5. Conclusion and future work

We have shown how we can use ZX-diagram simplification to improve quantum simulation methods based on stabiliser decompositions. We successfully simulated random 50- and 100-qubit Pauli exponential circuits with up to 70 T gates, as well as 50-qubit hidden shift circuits with up to 1400 T gates. For the former class of circuits, most of the benefit of our method comes from the initial T-count optimisation. This is particularly apparent for the 100-qubit circuits which are quite sparse, and hence where a lot more non-Clifford gates act on trivial states or effects. Our results hence make clear that for stabiliser-decomposition based simulation methods it is important to cancel as many non-Clifford gates in the original circuit. For the hidden shift circuits there is also an enormous benefit from cancelling T gates in the first optimisation step, but subsequent simplifications on the non-unitary diagrams also contribute significantly to the reduced simulation time of our method.

There are many areas where our method can potentially be improved. A natural next step is to incorporate the improved magic state decompositions presented in reference [39] into our method. This will involve finding convenient ZX-diagram presentations for the stabiliser states in these decompositions and running some further experiments.

Our method could also be improved by utilising a more powerful ZX-calculus simplification strategy. Our strategy is that of reference [29], which is specifically intended for unitary circuits, and as such does not include rewrites that only apply in a non-unitary context. In particular, we could use the *supplementarity* rule of reference [38] to cancel additional non-Clifford phases. We could also include additional simplification strategies to reduce the T-count of the diagram. While the strategies of references [3, 23] are probably too slow to apply to thousands of diagrams in parallel, the heuristics of reference [17] might be fast enough to apply to the first couple of layers of diagrams. For this to work best, we will need to find the best trade-off between the two (theoretically) exponential tasks of full T-count minimisation and stabiliser decomposition. It may be the case that, with a more powerful simplifier, we can switch to the more efficient T-count 4 representation (20) for CCZ gates without sacrificing nice simplification behaviour in the resulting ZX-diagrams.

In a similar vein, the simplification strategy seems to be quite sensitive to which blocks of non-Clifford spiders are chosen for decomposition as in equation (17). Our current implementation orders non-Clifford spiders (roughly) according to the time ordering of the T gates that produced them and applies

decompositions in order. This seems to work better empirically than any of the strategies we have tried based on choosing blocks at random, possibly because more simplifications can be found if the spiders being decomposed are relatively close to each other. However, more investigation is needed to see if this is the case or if better selection strategies can be employed to lower the overall number of terms.

Another approach to consider would be to directly represent the CCZ gates using the *H-boxes* of the *ZH-calculus* [6, 7]. A CCZ gate has a stabiliser rank of 2, so this could potentially lead to a significant reduction in the number of terms needed for simplification. Instead of the simplification strategy of section 2.3 based on local complementation, we could then use the strategy of reference [32] that uses *hyper-local-complementation*. However, this method has a tendency to increase the number of H-boxes, so care must be taken for this method to actually lead to a reduction in the number of terms required for simulation.

We have only considered exact strong simulation in this paper. In particular, our exact calculation of marginals doubles the T-count right at the beginning. This can be avoided by instead doing norm estimation using random stabiliser states sampled from a projective *t*-design [8]. However, in preliminary experiments, we found this method to be prohibitively slow at obtaining enough samples to approximate the norm to high precision. Part of the problem here is that, for calculating many independent amplitudes, ZX-calculus simplification seems to be quite a bit slower than a fast tableau simulator. Perhaps this could be improved by mixing ZX and tableau-based simulation or finding better ways to share some of the simplification work across all the samples.

Finally, it would be interesting to see whether techniques based on the stabiliser extent [8], rather than the stabiliser rank, can be accelerated by the ZX-calculus. In this approach, summands are sampled according to the relative magnitudes of their coefficients rather than computed exhaustively. The amount of samples needed to obtain a good approximation of the state then depends on the stabiliser extent, which effectively measures how much negativity is present in the list of coefficients. One possible avenue for incorporating ZX-calculus simplification is to adopt a hybrid approach. For example, one could perform the ZX-diagram decomposition we have described here up to a fixed depth, then approximate the value of each summand using random sampling based on the stabiliser extent.

In this work, we achieved substantial improvements on prior work by combining known techniques for classical simulation and optimisation of quantum circuits in a relatively simple way. The broader lesson here is that we should not treat classical simulation and optimisation as two separate problems, but rather as two complementary ways to extract information about the behaviour of a quantum circuit, which can be used together fruitfully.

Acknowledgments

AK is supported by AFOSR Grant FA2386-18-1-4028. JvdW is funded by a NWO Rubicon personal fellowship. The authors would like to thank Niel de Beaudrap, Mark Howard, Earl Campbell, and Padraic Calpin for useful discussions related to the stabiliser decompositions in [8].

Data availability statement

All data that support the findings of this study are included within the article (and any supplementary files).

ORCID iDs

Aleks Kissinger  <https://orcid.org/0000-0002-6090-9684>

John van de Wetering  <https://orcid.org/0000-0002-5405-8959>

References

- [1] 2021 *QuiZX: A Quick Rust Port of PyZX* (<https://github.com/Quantomatic/quixz>)
- [2] Aaronson S and Gottesman D 2004 Improved simulation of stabilizer circuits *Phys. Rev. A* **70** 052328
- [3] Amy M and Mosca M 2019 T-count optimization and Reed–Muller codes *IEEE Trans. Inf. Theory* **65** 4771–84
- [4] Arute F *et al* 2019 Quantum supremacy using a programmable superconducting processor *Nature* **574** 505–10
- [5] Backens M 2014 The ZX-calculus is complete for stabilizer quantum mechanics *New J. Phys.* **16** 093021
- [6] Backens M and Kissinger A 2019 Z H: a complete graphical calculus for quantum computations involving classical non-linearity *Proc. 15th Int. Conf. Quantum Physics and Logic* (Electronic Proc. Theoretical Computer Science vol 287) (Halifax, Canada 3–7 June 2018) ed P Selinger and G Chiribella (Open Publishing Association) pp 18–34

- [7] Backens M, Miller-Bakewell H, Kissinger A and van de Wetering J 2020 Completeness of the ZH-calculus (arXiv:2103.06610)
- [8] Bravyi S, Browne D, Calpin P, Campbell E, Gosset D and Howard M 2019 Simulation of quantum circuits by low-rank stabilizer decompositions *Quantum* **3** 181
- [9] Bravyi S and Gosset D 2016 Improved classical simulation of quantum circuits dominated by Clifford gates *Phys. Rev. Lett.* **116** 250501
- [10] Bravyi S and Gosset D 2016 Improved classical simulation of quantum circuits dominated by Clifford gates (arXiv:1601.07601)
- [11] Bravyi S, Smith G and Smolin J A 2016 Trading classical and quantum computational resources *Phys. Rev. X* **6** 021043
- [12] Coecke B and Duncan R 2008 Interacting quantum observables *Proc. 37th Int. Coll. Automata, Languages and Programming (ICALP)* (Lecture Notes in Computer Science)
- [13] Coecke B and Duncan R 2011 Interacting quantum observables: categorical algebra and diagrammatics *New J. Phys.* **13** 043016
- [14] Coecke B and Kissinger A 2017 *Picturing Quantum Processes* (Cambridge: Cambridge University Press)
- [15] Cowtan A, Dilkes S, Duncan R, Simmons W and Sivarajah S 2020 Phase gadget synthesis for shallow circuits *Proc. 16th Int. Conf. Quantum Physics and Logic* (Electronic Proc. Theoretical Computer Science vol 318) (Chapman University, Orange, CA, USA 10–14 June 2019) ed B Coecke and M Leifer (Open Publishing Association) pp 213–28
- [16] Cowtan A, Simmons W and Duncan R 2020 A generic compilation strategy for the unitary coupled cluster ansatz (arXiv:2007.10515)
- [17] de Beaudrap N, Bian X and Wang Q 2020 Techniques to reduce $\pi/4$ -parity-phase circuits, motivated by the ZX calculus *Proc. 16th Int. Conf. Quantum Physics and Logic* (Electronic Proc. Theoretical Computer Science vol 318) (Chapman University, Orange, CA, USA 10–14 June 2019) ed B Coecke and M Leifer (Open Publishing Association) pp 131–49
- [18] Duncan R, Kissinger A, Perdrix S and van de Wetering J 2020 Graph-theoretic simplification of quantum circuits with the ZX-calculus *Quantum* **4** 279
- [19] Gao X and Duan L 2018 Efficient classical simulation of noisy quantum computation (arXiv:1810.03176 [quant-ph])
- [20] Giles B and Selinger P 2013 Exact synthesis of multiqubit Clifford + T circuits *Phys. Rev. A* **87** 032332
- [21] Gosset D, Kliuchnikov V, Mosca M and Russo V 2014 An algorithm for the T-count *Quantum Inf. Comput.* **14** 1261–76
- [22] Hadzihanovic A, Ng K F and Wang Q 2018 Two complete axiomatisations of pure-state qubit quantum computing *Proc. 33rd Annual ACM/IEEE Symp. Logic in Computer Science, LICS'18* (New York: ACM) pp 502–11
- [23] Heyfron L E and Campbell E T 2018 An efficient quantum compiler that reduces T count *Quantum Sci. Technol.* **4** 015004
- [24] Huang C *et al* 2020 Classical simulation of quantum supremacy circuits (arXiv:2005.06787 [quant-ph])
- [25] Jeandel E, Simon P and Vilmart R 2018 A complete axiomatisation of the ZX-calculus for Clifford + T quantum mechanics *Proc. 33rd Annual ACM/IEEE Symp. Logic in Computer Science* (New York: ACM) pp 559–68
- [26] Jeandel E, Simon P and Vilmart R 2018 Diagrammatic reasoning beyond Clifford + T quantum mechanics *Proc. 33rd Annual ACM/IEEE Symp. Logic in Computer Science, LICS'18* (New York: ACM) pp 569–78
- [27] Jones C 2013 Low-overhead constructions for the fault-tolerant Toffoli gate *Phys. Rev. A* **87** 022328
- [28] Kissinger A and van de Wetering J 2020 PyZX: large scale automated diagrammatic reasoning *Proc. 16th Int. Conf. Quantum Physics and Logic* (Electronic Proc. Theoretical Computer Science vol 318) (Chapman University, Orange, CA, USA 10–14 June 2019) ed B Coecke and M Leifer (Open Publishing Association) pp 229–41
- [29] Kissinger A and van de Wetering J 2020 Reducing the number of non-Clifford gates in quantum circuits *Phys. Rev. A* **102** 022406
- [30] Kocia L 2020 Improved strong simulation of universal quantum circuits (arXiv:2012.11739 [quant-ph])
- [31] Kuijpers S, van de Wetering J and Kissinger A 2019 Graphical Fourier theory and the cost of quantum addition (arXiv:1904.07551)
- [32] Lemonnier L, van de Wetering J and Kissinger A 2020 Hypergraph simplification: linking the path-sum approach to the ZH-calculus (arXiv:2003.13564)
- [33] Litinski D 2019 A game of surface codes: large-scale quantum computing with lattice surgery *Quantum* **3** 128
- [34] Markov I L and Shi Y 2008 Simulating quantum computation by contracting tensor networks *SIAM J. Comput.* **38** 963–81
- [35] Nielsen M A and Chuang I L 2010 *Quantum Computation and Quantum Information* (Cambridge: Cambridge University Press)
- [36] Pan F and Zhang P 2021 Simulating the Sycamore quantum supremacy circuits (arXiv:2103.03074 [physics, physics:quant-ph])
- [37] Penrose R 1971 Applications of negative dimensional tensors *Combinatorial Mathematics and its Applications* (New York: Academic) pp 221–44
- [38] Perdrix S and Wang Q 2016 Suplementarity is necessary for quantum diagram reasoning *41st Int. Symp. Mathematical Foundations of Computer Science (MFCS 2016)* (Leibniz Int. Proc. Informatics (LIPIcs) vol 58) (Krakow, Poland) 76:1–76:14
- [39] Qassim H, Pashayan H and Gosset D 2021 Improved upper bounds on the stabilizer rank of magic states *Quantum* **5** 12
- [40] Rötteler M 2010 Quantum algorithms for highly non-linear Boolean functions *Proc. 21st Annual ACM-SIAM Symp. Discrete Algorithms* (Philadelphia, PA: SIAM) pp 448–57
- [41] Selinger P 2007 Dagger compact closed categories and completely positive maps *Electron. Notes Theor. Comput. Sci.* **170** 139–63
- [42] van de Wetering J 2020 ZX-calculus for the working quantum computer scientist (arXiv:2012.13966)
- [43] Vilmart R 2019 A near-minimal axiomatisation of ZX-calculus for pure qubit quantum mechanics *2019 34th Annual ACM/IEEE Symp. Logic in Computer Science (LICS)* pp 1–10